Post Graduate Government College
Sector 11 , Chandigarh

HeartBites

A FINAL YEAR MAJOR PROECT

Submitted By:-

Shikhar Sunyal
Roll No. 4023/23
BCA 3rd Year

Submitted To:-

Dr. Puneet Modgil

INDEX

S.No.	TOPIC	PAGE NO.	REMARKS
1	Project Title & About the Project	1	
2	Project Scope	2	
3	System Architecture	5	
4	Database Design	9	
5	Design of GUI	12	
6	Behavioral Logic	16	
7	Implementation	19	
8	Testing & QA	22	
9	Final Deployment	24	

Signature

ABOUT THE PROJECT

HeartBites – "A MERN Stack Social Media Application"

HeartBites is a full-stack social-media web application built using the MERN stack (MongoDB, Express.js, React.js, and Node.js). It aims to provide a smooth and engaging platform where users can discover potential matches based on their preferences. The application focuses on delivering a clean interface, strong performance, and a secure user experience while demonstrating real-world full-stack development.

The platform begins with a simple registration process where users sign up using their name, email, and password. After registration, they complete their profile by adding a profile picture, gender, date of birth (used to calculate age), and a short bio. Users also set their gender preference, which helps tailor the profile suggestions they receive.

A key feature of HeartBites is its swipe-based matching system. Users can like or pass on profiles, and when two users mutually like each other, a match is created. This ensures that only interested users can connect, making interactions more meaningful.

Once matched, users can communicate through a real-time chat system powered by Socket.io. Messages are displayed in a clean interface with timestamps, and conversations are stored in the database for future access. The application is deployed using Vercel (frontend), Render (backend), and MongoDB Atlas, making it fully functional and accessible online.

SCOPE THE PROJECT

Overview

HeartBites is a full-stack web-based dating application developed using the MERN stack. The project focuses on enabling users to create profiles, discover potential matches based on preferences, and communicate through a real-time chat system within a secure and responsive environment.

The primary objective of the HeartBites dating application is to develop a secure and reliable authentication system using JWT to ensure safe user access. The project aims to allow users to create and manage personalized profiles, enabling a more engaging and customized experience. It also focuses on implementing a swipe-based matching mechanism that allows users to express interest and form mutual connections. Additionally, the application provides real-time communication between matched users, enhancing interaction through instant messaging. Finally, the project is designed to be deployed as a fully functional web application, making it easily accessible to users over the internet.

Features of HeartBites

1. User Management-

- User registration and login
- Secure authentication using JWT and HTTP-only cookies
- Profile creation and update (photo, bio, gender, DOB, preferences)

2. Matching System-

- Gender-based user suggestions
- Swipe right (like) and swipe left (pass) functionality
- Mutual match creation system

3. Communication-

- Real-time chat using Socket.io
- Persistent message storage
- Online/offline user status tracking

4. User Experience-

- Interactive swipe animations
- Matches listing page
- Clean and responsive UI

5. Media Handling-

- Profile picture upload and storage using Cloudinary

Tech Stack

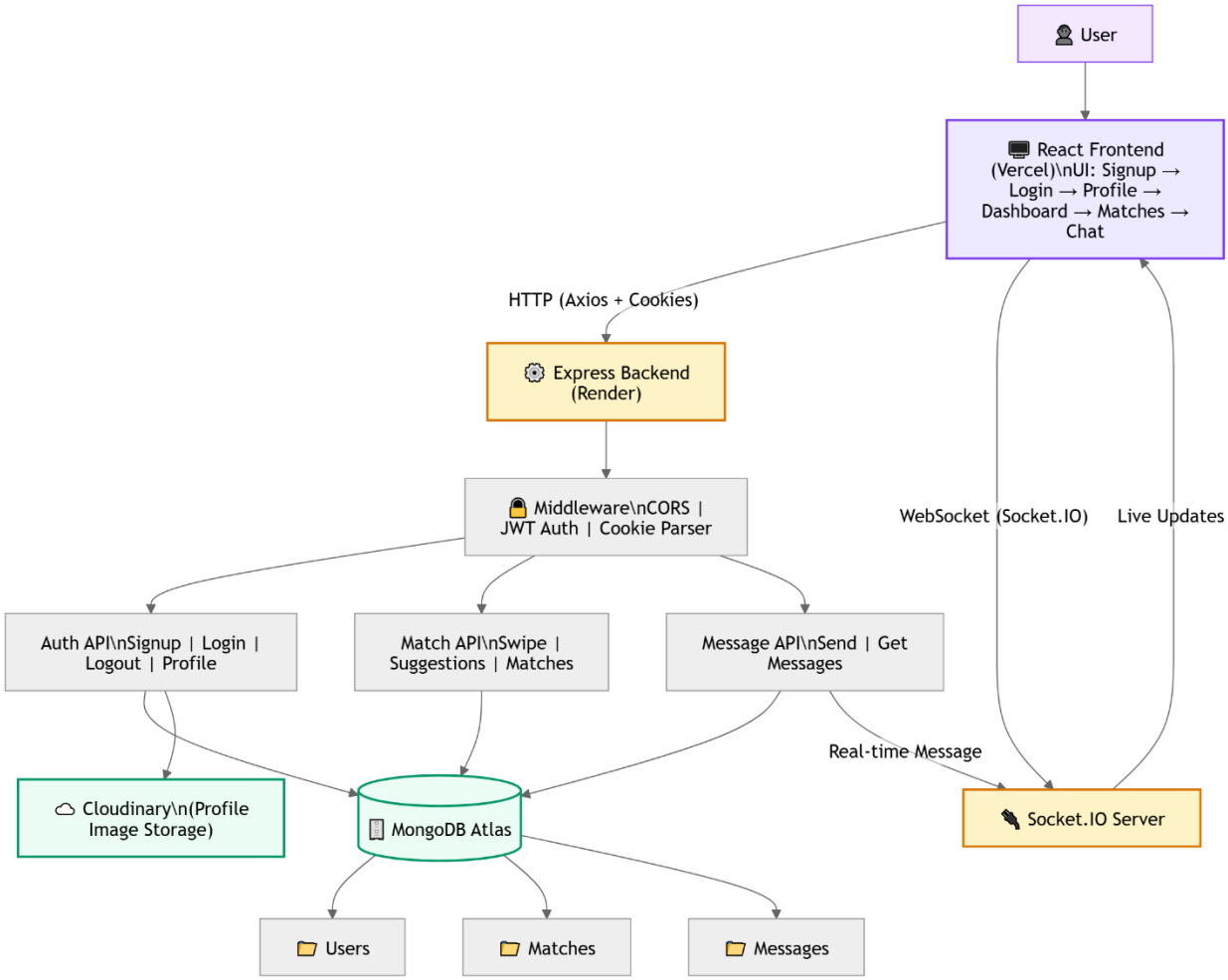
- **Frontend:** React.js, Tailwind CSS
- **Backend:** Node.js, Express.js
- **Database:** MongoDB (Atlas)
- **Authentication:** JWT
- **Real-time Communication:** Socket.io
- **Media Storage:** Cloudinary
- **Deployment:** Vercel (Frontend), Render (Backend)

Could Have Features

- Email/OTP verification
- Video or voice calling features

- Mobile application (Android/iOS)
- AI-based matchmaking
- Paid subscriptions and monetization features
- Push notifications
- Location-based matching

SYSTEM ARCHITECTURE



FlowChart

Overview

1. **User Layer**

- The system is initiated by the end user, who interacts with the application to perform activities such as registration, login, profile management, matching, and messaging.

2. **Presentation Layer (Frontend)**

- Developed using React.js and deployed on Vercel.
- Responsible for rendering the user interface and handling client-side interactions.
- Facilitates communication with the backend through:
  - HTTP requests (via Axios and cookies) for standard operations
  - WebSocket connections (via Socket.IO) for real-time features

3. **Application Layer (Backend)**

- Built using Node.js and Express.js, hosted on Render.
- Serves as the central processing unit, managing business logic and request handling.
- Incorporates middleware for:
  - Cross-Origin Resource Sharing (CORS)
  - JSON parsing and cookie handling
  - JWT-based authentication and route protection
- Provides modular API endpoints:
  - Authentication Module — handles user registration, login, logout, and profile updates
  - Matching Module — manages user suggestions, swipe actions, and match creation

- Messaging Module — enables sending and retrieving messages

4. **Real-Time Communication Layer**

- Implemented using Socket.IO.
- Enables bi-directional, real-time communication between client and server.
- Supports:
  - Instant message delivery
  - Live user status tracking
  - Event-based communication without page reload

5. **Data Layer (Database)**

- Utilizes MongoDB Atlas as a cloud-based NoSQL database.
- Maintains structured collections:
  - **Users** — stores authentication and profile data
  - **Matches** — records mutual user connections
  - **Messages** — stores chat history

6. **External Service Layer**

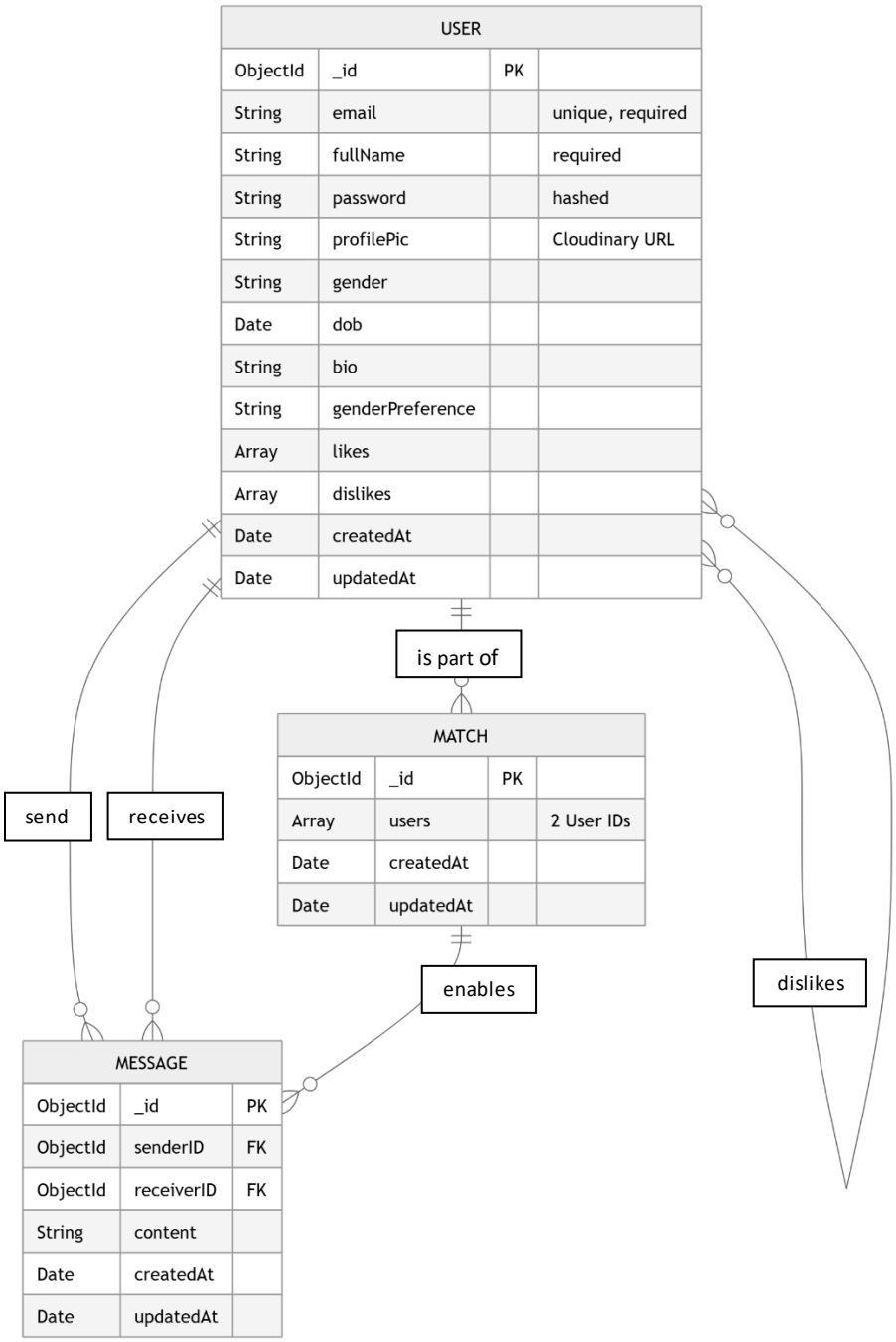
The application integrates Cloudinary as an external service for managing user profile images. It handles the uploading, storage, and optimized delivery of images through a cloud-based CDN, ensuring efficient performance and reduced load on the main server.

7. **System Flow Summary**

The overall system flow begins with the user interacting with the frontend interface, which sends requests to the backend using HTTP

protocols and WebSocket connections. The backend processes these requests, applies the necessary business logic, and communicates with the database and external services to retrieve or store data. For real-time features such as messaging, Socket.IO enables instant communication between users by pushing updates directly to the frontend, ensuring a smooth and responsive user experience.

DATABASE DESIGN



ER-Diagram

The HeartBites application uses a MongoDB-based NoSQL database design that is optimized for scalability, flexibility, and efficient handling of user interactions. The database is divided into three main collections: **User**, **Match**, and **Message:**

1. User Collection

The User collection stores all information related to individual users, including both authentication and profile data. It contains fields such as email, password (hashed), full name, gender, date of birth, bio, and profile picture. Additionally, it stores the gender preference to filter user suggestions.

- Maintains likes and dislikes arrays (references to other users)
- Stores complete profile and authentication details
- Supports user personalization and interaction tracking

2. Match Collection

The Match collection stores information about mutual connections between users. A match is created only when two users like each other.

- Contains two user IDs representing the matched pair
- Includes timestamps for tracking match creation
- Stores only valid mutual matches

3. Message Collection

The Message collection stores all chat messages exchanged between users. It ensures that communication is persistent and can be accessed anytime.

- Contains sender ID and receiver ID

- Stores message content and timestamps
- Maintains complete chat history

Relationships Between Collections

The three collections are interconnected to support the core functionalities of the application.

- **User to User (Self Relationship)**
  - Implemented through likes and dislikes arrays
  - Enables swipe-based interaction and filtering
- **User to Match**
  - A user can be part of multiple matches
  - The Match collection links two users, resolving the many-to-many relationship
- **User to Message**
  - A user can send and receive multiple messages
  - Establishes a one-to-many relationship
- **Match to Message**
  - Messages are exchanged only after a match is created
  - The Match collection acts as a bridge enabling communication

DESIGN OF GUI

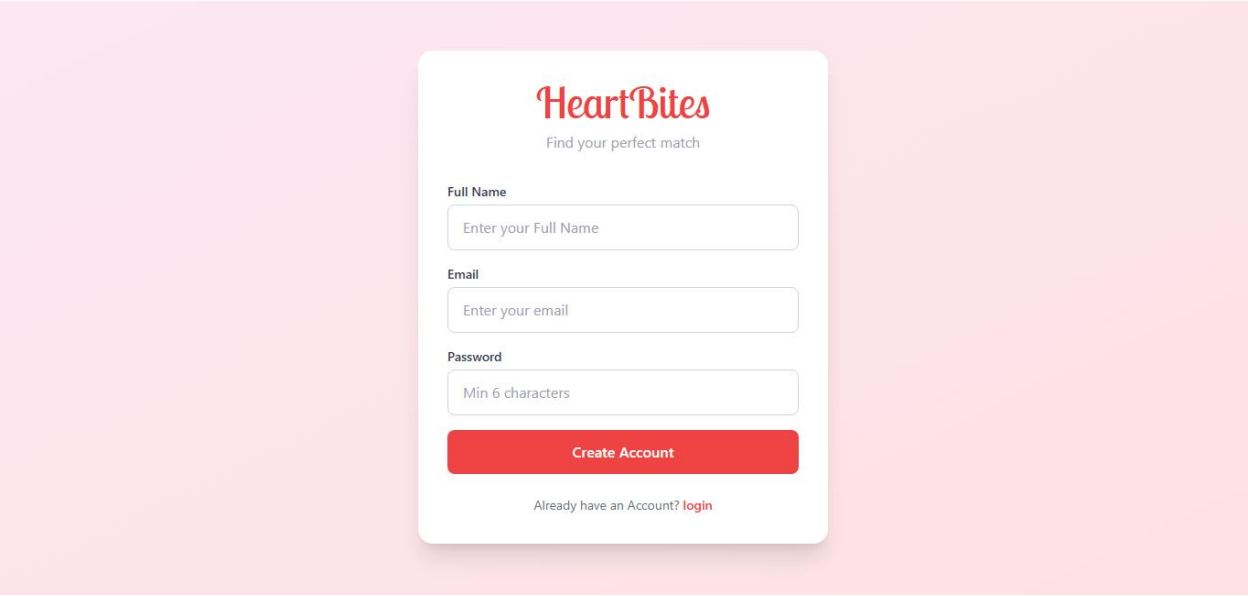
The graphical user interface of HeartBites has been designed with a focus on simplicity, elegance, and user experience. The overall design philosophy follows a **"Minimalist"** aesthetic with a warm off-white and cream background tone complemented by deep purple accents, giving the application a modern and premium feel. The Lobster font is used for the HeartBites brand name across all pages, providing a distinctive and memorable visual identity.

Design Consistency

All pages across the application maintain a consistent design language with the following shared elements — the **Lobster font** for the brand name, deep purple as the primary accent color, cream off-white as the background color, rounded corners on all cards and buttons, and smooth hover and transition animations on interactive elements. This consistency creates a cohesive and professional user experience throughout the entire application.

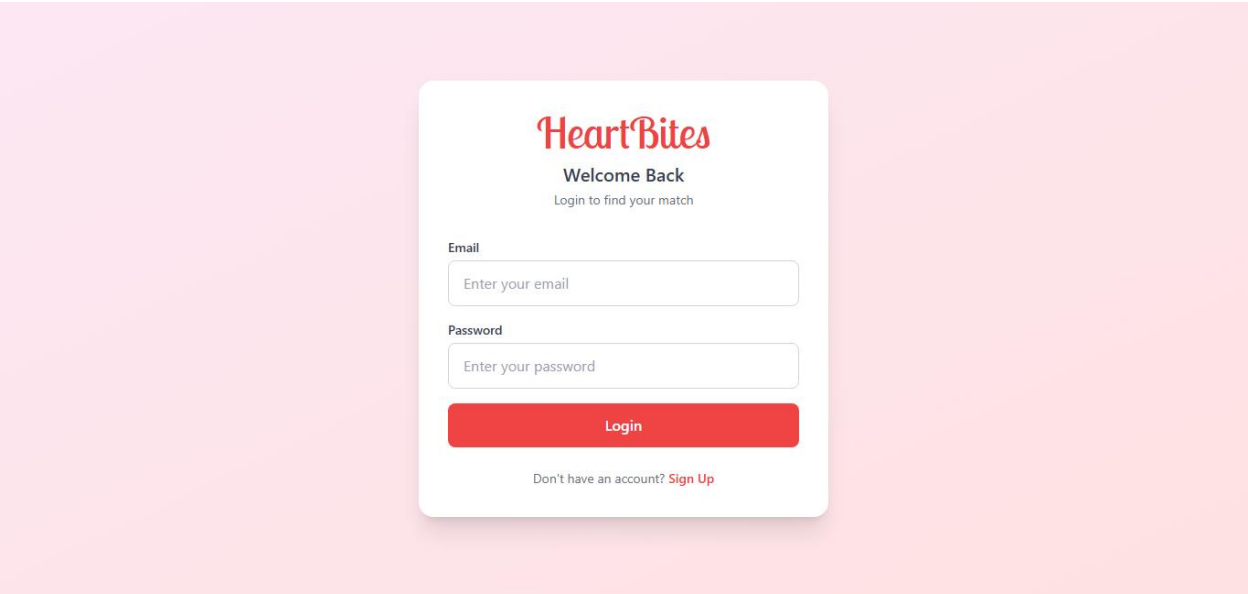
Signup Page

Allows new users to create an account by entering basic details such as name, email, and password. Ensures validation and secure registration.



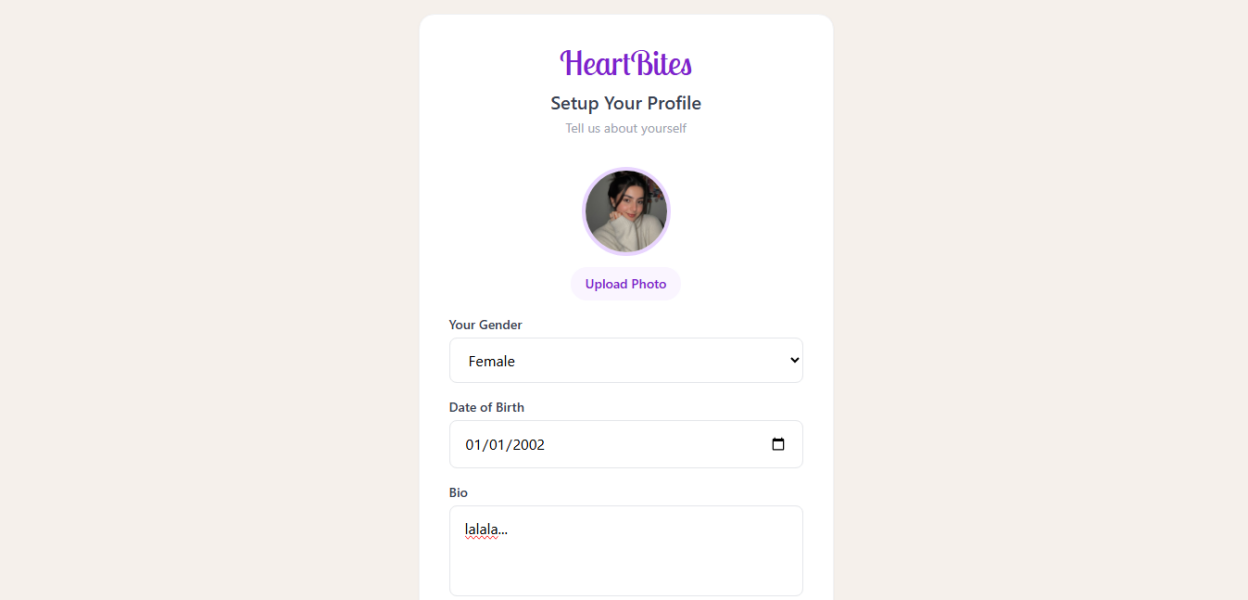
Login Page

Enables existing users to authenticate using their credentials and access the application securely via JWT-based sessions.



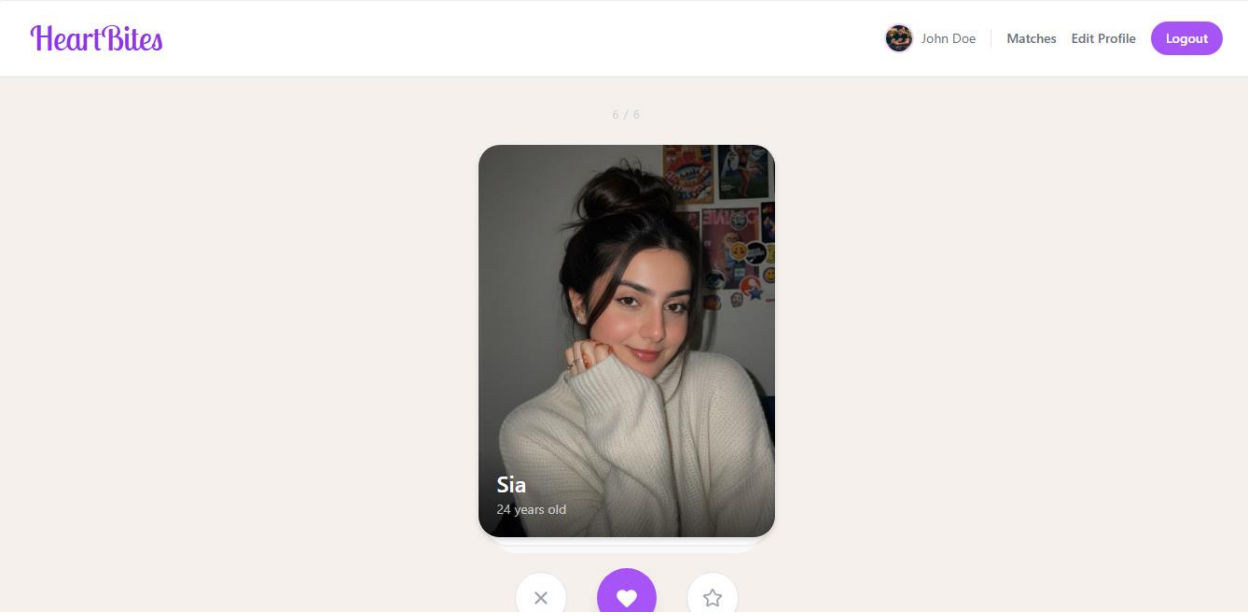
Profile Setup Page

Lets users complete their profile by adding personal details like profile picture, bio, gender, date of birth, and preferences.



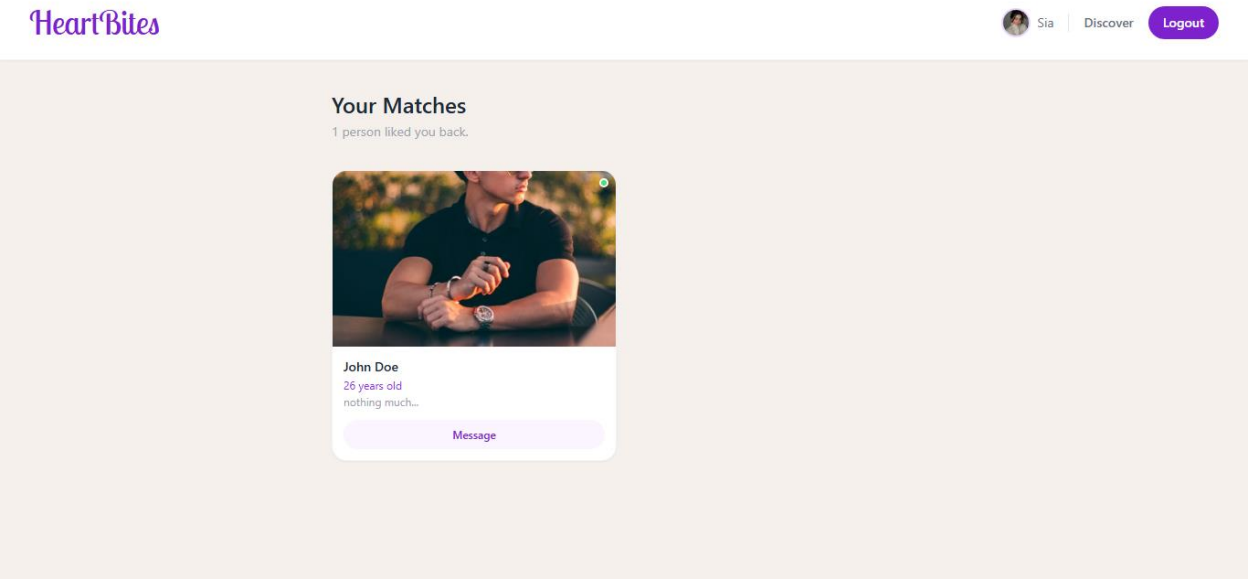
Dashboard Page

Displays user suggestions in a card-based layout where users can swipe right (like) or left (dislike) to find potential matches.



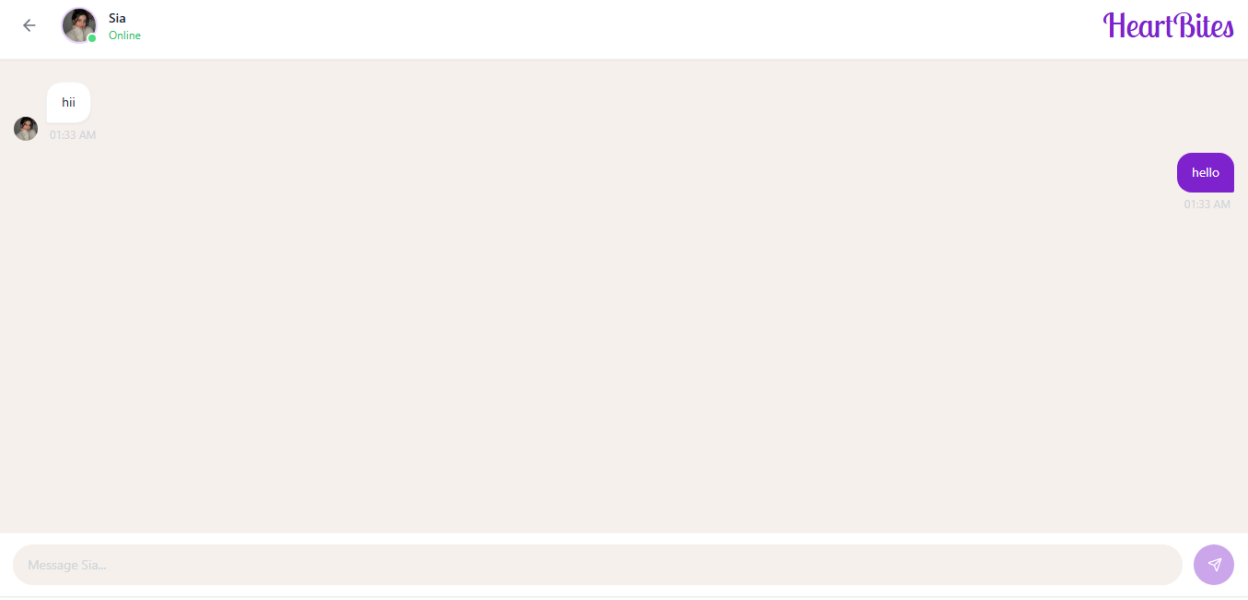
Matches Page

Shows a list of users with whom mutual matches have been established, allowing easy navigation to conversations.

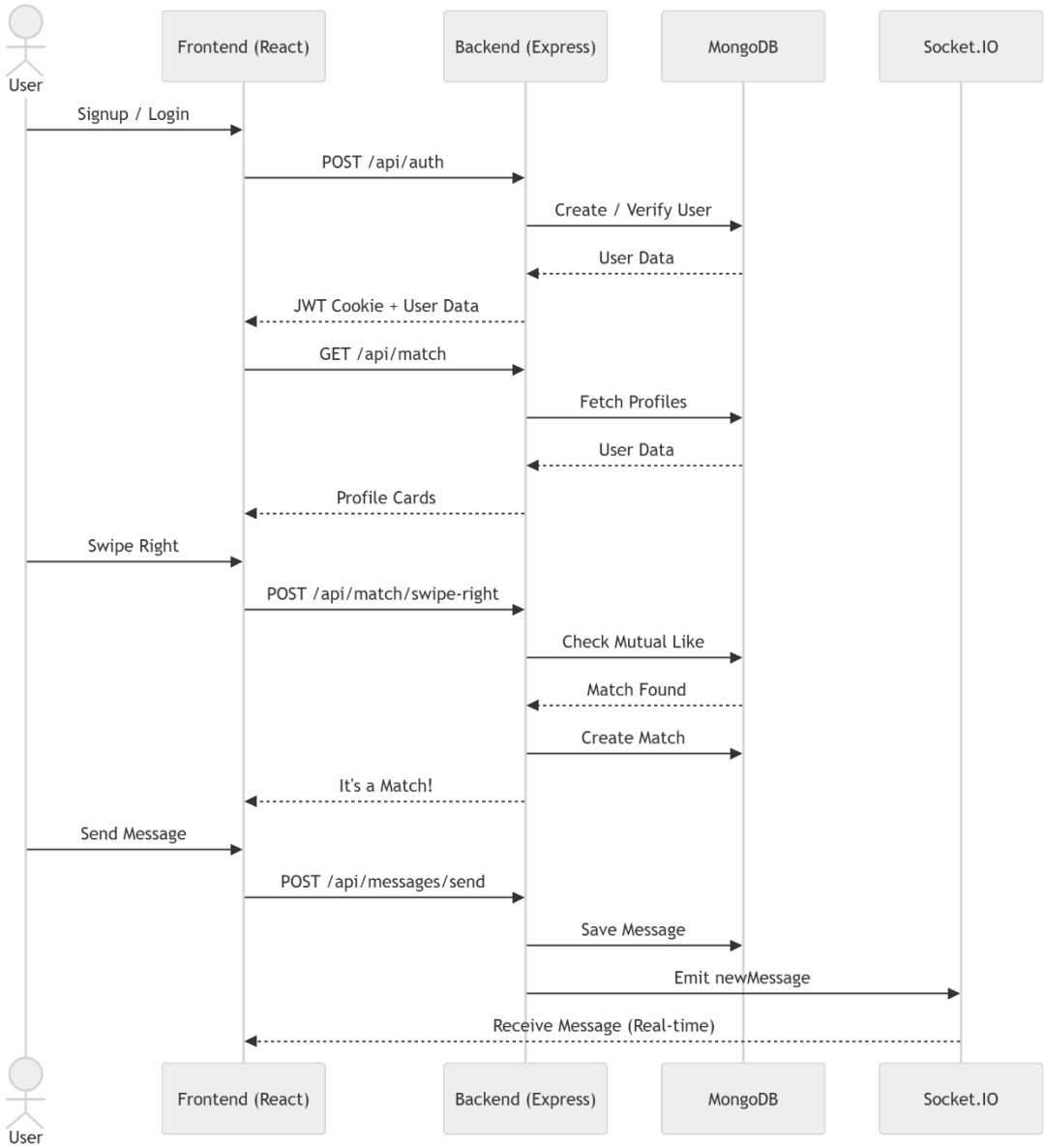


ChatPage

Provides a real-time messaging interface where matched users can send and receive messages instantly using Socket.IO.



BEHAVIORAL LOGIC



Sequential-Diagram

The sequence diagram represents the interaction between the **User**, **Frontend (React)**, **Backend (Express)**, **Database (MongoDB)**, and **Socket.IO** for real-time communication. It outlines the complete workflow of the application, divided into three major phases:

1. Authentication Phase

The sequence begins with the **authentication process**, where the user signs up or logs in through the frontend. The request is sent to the backend, which interacts with the database to create or verify the user. Upon successful authentication, a JWT token along with user data is returned to the client, enabling secure access to the application.

2. Matching Phase

In the **matching phase**, the frontend requests user profiles from the backend, which retrieves filtered data from the database and displays it as profile cards. When a user performs a swipe action (like), the backend checks for a mutual like. If both users have liked each other, a match is created and stored, and the user is notified accordingly.

3. Messaging (Real-Time Chat) Phase

The **messaging phase** is activated after a successful match. When a user sends a message, it is sent to the backend and stored in the database for persistence. Simultaneously, the backend uses Socket.IO to deliver the message in real time to the recipient, ensuring instant communication without page refresh.

Overall System Behavior

- Ensures secure authentication using JWT.
- Provides efficient matchmaking through mutual likes.
- Enables real-time communication using WebSockets (Socket.IO).
- Maintains data persistence through MongoDB.

IMPLEMENTATION

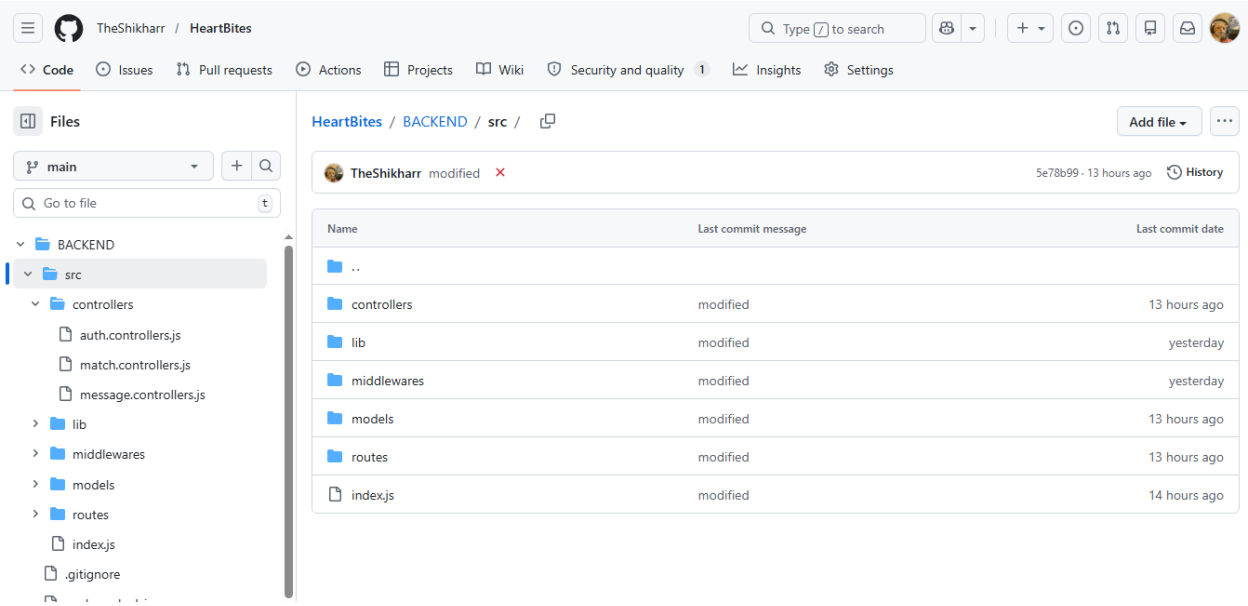
The implementation of HeartBites is structured as a full-stack web application using modern technologies to deliver a responsive and real-time matchmaking platform. The frontend is built with React.js, utilizing Zustand for state management and Axios for API communication, providing an interactive interface for authentication, profile management, matching, and chat features.

The backend is developed using Node.js and Express, handling core functionalities such as user authentication, swipe-based matching, and messaging through RESTful APIs. Security is ensured using JWT authentication stored in HTTP-only cookies. MongoDB Atlas is used as the database to manage users, matches, and messages, while Cloudinary handles profile image storage.

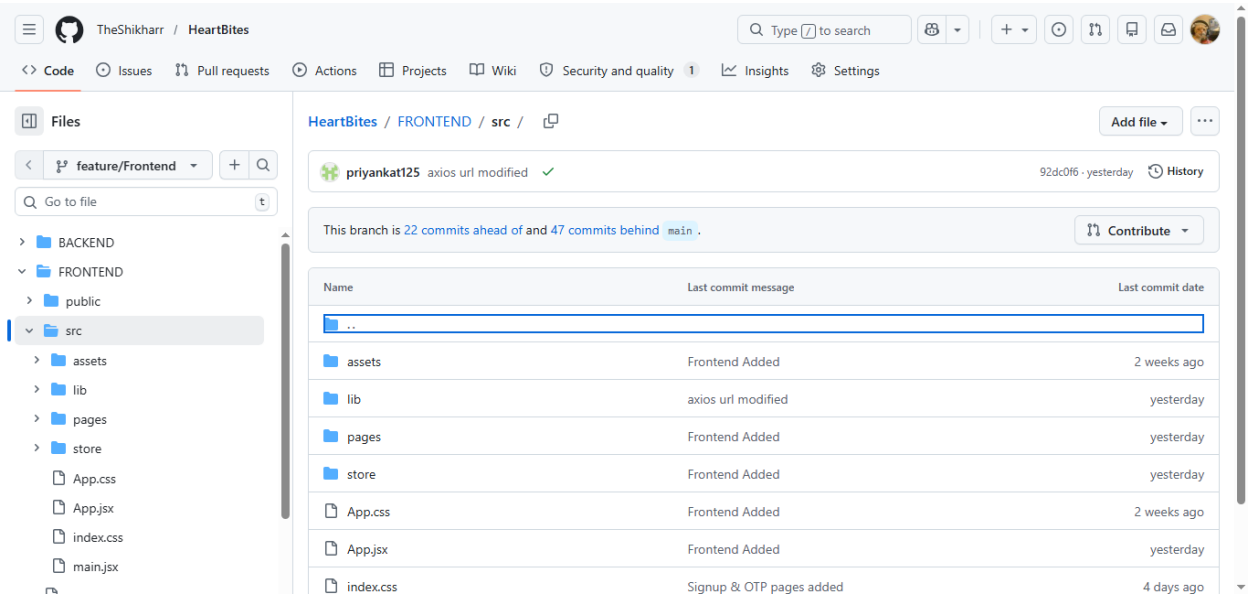
Real-time communication is implemented using Socket.IO, enabling instant messaging and live updates without page refresh. The project is managed using Git and GitHub for version control, and deployed on platforms like Vercel (frontend) and Render (backend), ensuring scalability and reliability.

GitHub Repository

BACKEND:



FRONTEND:



GitHub Links

BACKEND LINK:

main-branch:
https://github.com/TheShikharr/HeartBites/tree/main/BACKEND

FONTEND LINK:

main-branch:
https://github.com/TheShikharr/HeartBites/tree/main/FRONTEND

feature/FRONTEND-branch:
https://github.com/TheShikharr/HeartBites/tree/feature/Frontend/FRONTEND

TESTING & QA

The testing and quality assurance of HeartBites focus on ensuring reliability, security, performance, and a seamless user experience across all features of the application.

Test Cases:

TEST CASE NO.	MODULE	TEST CASE	EXPECTED RESULT	STATUS
1	Authentication	Signup with valid data	Account created successfully, JWT cookie generated	Pass
2	Authentication	Signup with duplicate email	Error 400: Email already exists	Pass
3	Authentication	Login with correct credentials	Status 200, user data returned	Pass
4	Authentication	Login with incorrect password	Error 400: Invalid Credentials	Pass
5	Authentication	Access protected route without token	Error 401 Unauthorized	Pass
6	Profile	Update profile with image upload	Image uploaded to Cloudinary, profile updated	Pass
7	Matching	Fetch filtered user feed	Excludes self, liked, disliked,	Pass

			matched users	
8	Matching	Swipe right with mutual like	Match created, isMatch: true	Pass
9	Matching	Swipe left on a user	User added to dislikes, removed from feed	Pass
10	Messaging	Send message to matched user	Message stored and delivered	Pass
11	Messaging	Send message to unmatched user	Error 403 Forbidden	Pass
12	Messaging	Real-time messaging via Socket.IO	Receiver gets message instantly	Pass

FINAL DEPLOYMENT

HeartBites is successfully deployed and accessible as a fully functional web application.

Production Deployment

- **Frontend** is hosted on Vercel: https://heartbites.vercel.app
- **Backend API** is deployed on Render: https://heartbites-backend.onrender.com
- **Database** is managed using MongoDB Atlas (cloud-based).
- **Media Storage** is handled via Cloudinary for profile images.

Local Development Setup

- Backend server runs on: http://localhost:8080
- Frontend runs on: http://localhost:5173 (via Vite development server)
- Enables efficient testing and development before deployment.

Version Control & Repository

- The complete source code is hosted on GitHub: https://github.com/TheShikharr/HeartBites
- Managed using Git with:
  - Incremental commits
  - Clear development history
  - Separation of frontend and backend codebases